

MODUL 13

Nama: Ryan Akeyla Novianto Widodo

Kelas: 12 IF 07

NIM : 103112400081

DASAR TEORI

Dasar Teori C++

Diambil dari internet dan dirangkum ulang oleh saya dan dibantu dengan AI.

1. Inti Program

- Syntax: Mirip bahasa Inggris, tapi huruf besar dan kecil beda arti. Setiap perintah biasanya diakhiri dengan titik koma (;).

- Komentar: Catatan di kode yang tidak dijalankan.

- Satu baris: // Ini komentar

- Banyak baris: /* Ini komentar yang panjang */

- Header: File yang berisi daftar perintah dan fungsi yang bisa dipakai. Contoh:

cpp

```
#include <iostream> // Untuk perintah input/output (cout, cin)
```

- main() : Fungsi utama, tempat program mulai berjalan.

cpp

```
int main() {
```

```
    // Kode program di sini
```

```
    return 0; // Menandakan program selesai dengan benar
```

```
}
```

- Namespace: Seperti "area" untuk nama-nama perintah. std adalah namespace standar C++.

cpp

```
using namespace std; // Supaya tidak perlu tulis std::cout, std::cin, dll.
```

- Input/Output:

- cout : Menampilkan sesuatu di layar.

- cin : Membaca input dari keyboard.

cpp

```
cout << "Halo!" << endl; // Menampilkan "Halo!"
```

```
int umur;
```

```
cin >> umur; // Membaca angka dari keyboard dan disimpan di variabel "umur"
```

2. Jenis Data

- Integer (Bilangan Bulat):

- int : Bilangan bulat (contoh: -10, 0, 25).

- short : Bilangan bulat pendek.

- long : Bilangan bulat panjang.

- Floating-Point (Bilangan Desimal):

- float : Bilangan desimal (contoh: 3.14).

- double : Bilangan desimal dengan ketelitian lebih tinggi.

- Character (Karakter):

- char : Satu huruf, angka, atau simbol (contoh: 'a', 'Z', '5').

- Boolean (Logika):

- bool : Nilai benar (true) atau salah (false).

- void : "Kosong". Dipakai untuk fungsi yang tidak menghasilkan nilai.

3. Variabel

- Deklarasi: Memberi tahu program nama dan jenis data variabel.

cpp

```
int umur; // Variabel bernama "umur" untuk menyimpan bilangan bulat
```

```
double harga; // Variabel bernama "harga" untuk menyimpan bilangan desimal
```

- Inisialisasi: Memberi nilai awal ke variabel.

cpp

```
int umur = 20; // "umur" diisi dengan nilai 20
```

```
double harga = 99.50; // "harga" diisi dengan nilai 99.50
```

4. Operator

- Aritmatika: + (tambah), - (kurang), * (kali), / (bagi), % (sisanya).
- Penugasan: = (mengisi nilai ke variabel).
- Perbandingan: == (sama dengan), != (tidak sama dengan), > (lebih besar), < (lebih kecil), >= (lebih besar atau sama dengan), <= (lebih kecil atau sama dengan).
- Logika: && (DAN), || (ATAU), ! (TIDAK).
- Increment/Decrement: ++ (tambah 1), -- (kurang 1).

5. Struktur Kontrol

- if : Melakukan sesuatu jika kondisi benar.

cpp

```
if (umur >= 17) {  
    cout << "Anda sudah dewasa." << endl;  
} else {  
    cout << "Anda belum dewasa." << endl;  
}
```

- switch : Memilih salah satu dari beberapa pilihan.

cpp

```
int hari = 3;  
switch (hari) {  
    case 1:  
        cout << "Senin" << endl;  
        break;  
    case 2:
```

```

        cout << "Selasa" << endl;

        break;

    case 3:

        cout << "Rabu" << endl;

        break;

    default:

        cout << "Hari lain" << endl;

}

```

- for : Melakukan sesuatu berulang kali sebanyak yang ditentukan.

cpp

```

for (int i = 0; i < 5; i++) {

    cout << i << " "; // Menampilkan angka 0 sampai 4

}

cout << endl;

```

- while : Melakukan sesuatu berulang kali selama kondisi benar.

cpp

```

int angka = 1;

while (angka <= 3) {

    cout << angka << " ";

    angka++;

}

cout << endl;

```

- do-while : Sama seperti while , tapi dilakukan minimal sekali.

cpp

```

int pilihan;

do {

    cout << "Pilih 1, 2, atau 3: ";

```

```
cin >> pilihan;  
} while (pilihan < 1 || pilihan > 3);
```

6. Fungsi

- Definisi: Sekumpulan kode yang melakukan tugas tertentu.

cpp

```
int tambah(int a, int b) {  
    return a + b;  
}
```

- Pemanggilan: Menggunakan fungsi.

cpp

```
int hasil = tambah(5, 2); // Memanggil fungsi "tambah" dengan angka 5 dan 2  
cout << "Hasil: " << hasil << endl; // Menampilkan "Hasil: 7"
```

7. Pointer

- Definisi: Variabel yang menyimpan alamat memori variabel lain. (Agak rumit, tapi penting).

cpp

```
int umur = 25;  
  
int *p_umur = &umur; // p_umur menyimpan alamat memori dari variabel umur
```

DASAR TEORI MULTI LINKED LIST

Pengertian Multi Linked List (C++)

Multi Linked List adalah pengembangan dari struktur data linked list di mana satu node dapat memiliki lebih dari satu pointer untuk menghubungkannya dengan node lain. Dalam bahasa pemrograman C++, multi linked list digunakan untuk merepresentasikan hubungan data yang kompleks, seperti relasi satu-ke-banyak atau banyak-ke-banyak.

Berbeda dengan single linked list yang hanya memiliki satu pointer (next), pada multi linked list setiap node dapat memiliki beberapa pointer (misalnya next, child, prev, atau pointer ke struktur lain). Hal ini memungkinkan pembentukan struktur data seperti graph, tree, atau relasi antar entitas.

Multi linked list banyak digunakan pada aplikasi seperti sistem basis data, struktur organisasi, jaringan, dan manajemen relasi data.

Guided

```
#include <iostream>
#include <string>
using namespace std;

struct ChildNode
{
    string info;
    ChildNode *next;
};

struct ParentNode
{
    string info;
    ChildNode *childHead;
    ParentNode *next;
};

ParentNode *createParent(string info)
{
    ParentNode *newNode = new ParentNode;
    newNode->info = info;
    newNode->childHead = NULL;
    newNode->next = NULL;
    return newNode;
}

ChildNode *createChild(string info)
{
    ChildNode *newNode = new ChildNode;
    newNode->info = info;
    newNode->next = NULL;
    return newNode;
}

void insertParent(ParentNode *&head, string info)
{
    ParentNode *newNode = createParent(info);
    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        ParentNode *temp = head;
        while (temp->next != NULL)
        {
            temp = temp->next;
        }
    }
}
```

```

    temp->next = newNode;
}
}

void insertChild(ParentNode *head, string parentInfo, string childInfo)
{
    ParentNode *p = head;
    while (p != NULL && p->info != parentInfo)
    {
        p = p->next;
    }
    if (p != NULL)
    {
        ChildNode *newChild = createChild(childInfo);
        if (p->childHead == NULL)
        {
            p->childHead = newChild;
        }
        else
        {
            ChildNode *c = p->childHead;
            while (c->next != NULL)
            {
                c = c->next;
            }
            c->next = newChild;
        }
    }
}

void printAll(ParentNode *head)
{
    ParentNode *p = head;
    while (p != NULL)
    {
        cout << p->info;
        ChildNode *c = p->childHead;
        if (c != NULL)
        {
            while (c != NULL)
            {
                cout << " -> " << c->info;
                c = c->next;
            }
        }
        cout << endl;
        p = p->next;
    }
}

```

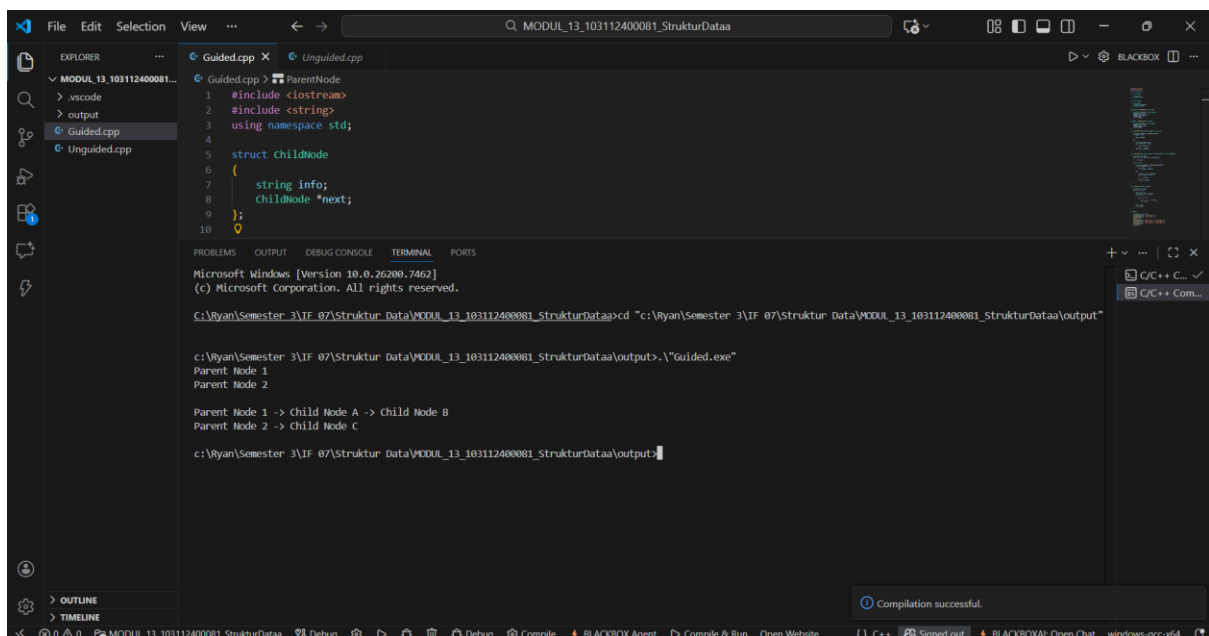


```

int main()
{
    ParentNode *list = NULL;
    insertParent(list, "Parent Node 1");
    insertParent(list, "Parent Node 2");
    printAll(list);
    cout << "\n";
    insertChild(list, "Parent Node 1", "Child Node A");
    insertChild(list, "Parent Node 1", "Child Node B");
    insertChild(list, "Parent Node 2", "Child Node C");
    printAll(list);
    return 0;
}

```

Hasil Coding:



```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 struct ChildNode
6 {
7     string info;
8     ChildNode *next;
9 };
10

```

Microsoft Windows [Version 10.0.26280.7462]
(c) Microsoft Corporation. All rights reserved.

c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL_13_103112400081_StrukturData\output> cd "c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL_13_103112400081_StrukturData\output"

c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL_13_103112400081_StrukturData\output> .\Guided.exe

Parent Node 1
Parent Node 2

Parent Node 1 -> Child Node A -> Child Node B
Parent Node 2 -> Child Node C

c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL_13_103112400081_StrukturData\output>

Compilation successful.

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita membuat dan mengimplementasikan struktur data Multi Linked List menggunakan bahasa pemrograman C++. Program ini dirancang untuk mengelola hubungan data bertipe satu-ke-banyak (one-to-many), di mana satu parent node dapat memiliki banyak child node.

Program ini memungkinkan pengguna untuk menambahkan parent node, menambahkan child node pada parent tertentu, serta menampilkan seluruh isi struktur multi linked list. Dengan menggunakan pointer dan linked list, program ini membantu memahami konsep relasi antar data, pengelolaan memori dinamis, serta struktur data non-linear dalam C++.

Unguided 1

Coding:

```
#include <iostream>
using namespace std;

typedef int infotipeinduk;
typedef int infotipeanak;

struct elemen_list_anak;
struct elemen_list_induk;

typedef elemen_list_induk* address_induk;
typedef elemen_list_anak* address_anak;

struct listanak {
    address_anak first;
    address_anak last;
};

struct elemen_list_anak {
    infotipeanak info;
    address_anak next;
    address_anak prev;
};

struct listinduk {
    address_induk first;
    address_induk last;
};

struct elemen_list_induk {
    infotipeinduk info;
    listanak anak;
    address_induk next;
    address_induk prev;
};

void CreateListInduk(listinduk &L) {
    L.first = NULL;
    L.last = NULL;
}

void CreateListAnak(listanak &L) {
    L.first = NULL;
    L.last = NULL;
}

address_induk alokasiInduk(infotipeinduk X) {
    address_induk P = new elemen_list_induk;
```

```

P->info = X;
CreateListAnak(P->anak);
P->next = NULL;
P->prev = NULL;
return P;
}

void dealokasiInduk(address_induk P) {
    delete P;
}

address_anak alokasiAnak(infotipeanak X) {
    address_anak P = new elemen_list_anak;
    P->info = X;
    P->next = NULL;
    P->prev = NULL;
    return P;
}

void dealokasiAnak(address_anak P) {
    delete P;
}

address_induk findElmInduk(listinduk L, infotipeinduk X) {
    address_induk P = L.first;
    while (P != NULL) {
        if (P->info == X) return P;
        P = P->next;
    }
    return NULL;
}

address_anak findElmAnak(listanak L, infotipeanak X) {
    address_anak P = L.first;
    while (P != NULL) {
        if (P->info == X) return P;
        P = P->next;
    }
    return NULL;
}

void insertFirstInduk(listinduk &L, address_induk P) {
    if (L.first == NULL) {
        L.first = L.last = P;
    } else {
        P->next = L.first;
        L.first->prev = P;
        L.first = P;
    }
}

```

```

void insertLastInduk(listinduk &L, address_induk P) {
    if (L.first == NULL) {
        insertFirstInduk(L, P);
    } else {
        L.last->next = P;
        P->prev = L.last;
        L.last = P;
    }
}

```

```

void insertAfterInduk(listinduk &L, address_induk Prec, address_induk P) {
    if (Prec->next == NULL) {
        insertLastInduk(L, P);
    } else {
        P->next = Prec->next;
        Prec->next->prev = P;
        Prec->next = P;
        P->prev = Prec;
    }
}

```

```

void insertFirstAnak(listanak &L, address_anak P) {
    if (L.first == NULL) {
        L.first = L.last = P;
    } else {
        P->next = L.first;
        L.first->prev = P;
        L.first = P;
    }
}

```

```

void insertLastAnak(listanak &L, address_anak P) {
    if (L.first == NULL) {
        insertFirstAnak(L, P);
    } else {
        L.last->next = P;
        P->prev = L.last;
        L.last = P;
    }
}

```

```

void insertAfterAnak(listanak &L, address_anak Prec, address_anak P) {
    if (Prec->next == NULL) {
        insertLastAnak(L, P);
    } else {
        P->next = Prec->next;
        Prec->next->prev = P;
        Prec->next = P;
        P->prev = Prec;
    }
}

```

```

    }
}

void deleteFirstInduk(listinduk &L, address_induk &P) {
    if (L.first == NULL) {
        P = NULL;
    } else if (L.first == L.last) {
        P = L.first;
        L.first = L.last = NULL;
    } else {
        P = L.first;
        L.first = P->next;
        L.first->prev = NULL;
    }
}

```

```

void deleteFirstAnak(listanak &L, address_anak &P) {
    if (L.first == NULL) {
        P = NULL;
    } else if (L.first == L.last) {
        P = L.first;
        L.first = L.last = NULL;
    } else {
        P = L.first;
        L.first = P->next;
        L.first->prev = NULL;
    }
}

```

```

int nBelas(listinduk L) {
    int count = 0;
    address_induk P = L.first;
    while (P != NULL) {
        count++;
        P = P->next;
    }
    return count;
}

```

```

int nBilas(listanak L) {
    int count = 0;
    address_anak P = L.first;
    while (P != NULL) {
        count++;
        P = P->next;
    }
    return count;
}

```

```

void printInfoAnak(listanak L) {

```

```

    address_anak P = L.first;
    while (P != NULL) {
        cout << P->info << " ";
        P = P->next;
    }
}

void printInfoInduk(listinduk L) {
    address_induk P = L.first;
    while (P != NULL) {
        cout << P->info << " : ";
        printInfoAnak(P->anak);
        cout << endl;
        P = P->next;
    }
}

int main() {
    listinduk LI;
    CreateListInduk(LI);

    insertFirstInduk(LI, alokasiInduk(10));
    insertLastInduk(LI, alokasiInduk(20));
    insertLastInduk(LI, alokasiInduk(30));

    address_induk parent = findElmInduk(LI, 20);
    if (parent != NULL) {
        insertLastAnak(parent->anak, alokasiAnak(101));
        insertLastAnak(parent->anak, alokasiAnak(102));
        insertLastAnak(parent->anak, alokasiAnak(103));
    }

    parent = findElmInduk(LI, 10);
    if (parent != NULL) {
        insertLastAnak(parent->anak, alokasiAnak(999));
    }

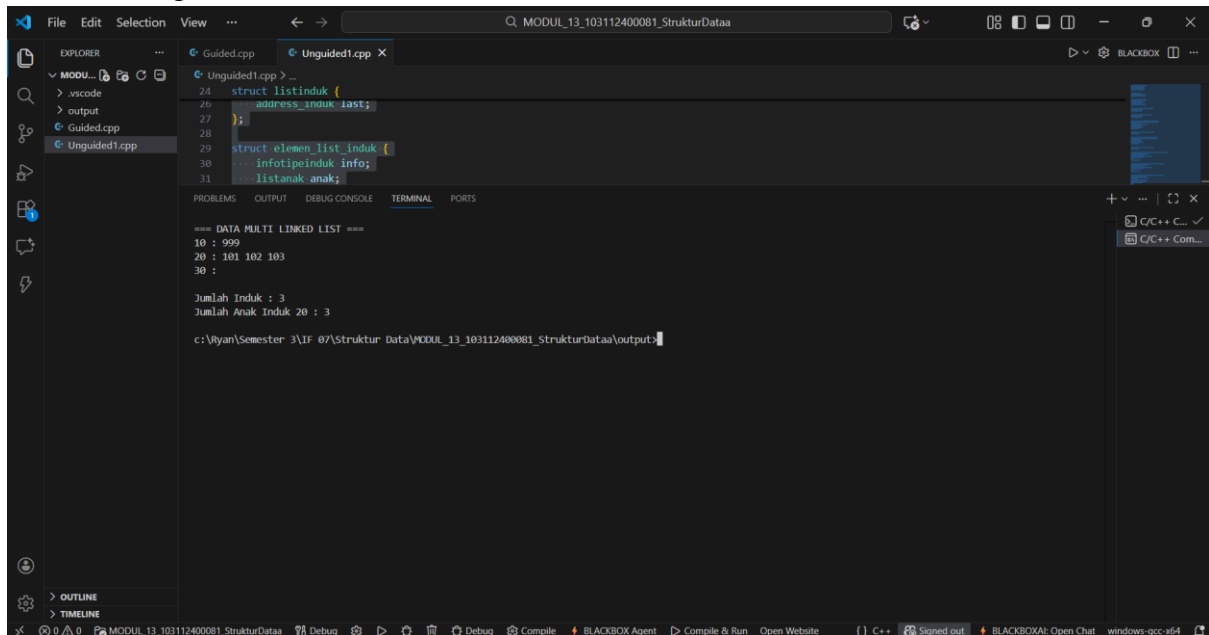
    cout << "=== DATA MULTI LINKED LIST ===" << endl;
    printInfoInduk(LI);

    cout << "\nJumlah Induk : " << nBelas(LI) << endl;
    if (findElmInduk(LI, 20) != NULL) {
        cout << "Jumlah Anak Induk 20 : "
            << nBilas(findElmInduk(LI, 20)->anak) << endl;
    }

    return 0;
}

```

Hasil Coding



The screenshot shows a C++ IDE with a file named `Unguided1.cpp` open. The code defines two structures: `listinduk` and `elemen_list_induk`. `listinduk` has a pointer to the next `listinduk` and a pointer to the first `elemen_list_induk`. `elemen_list_induk` has a pointer to the next `elemen_list_induk` and a pointer to its parent `listinduk`. The terminal output shows the following data:

```
=== DATA MULTI LINKED LIST ===
10 : 999
20 : 101 102 103
30 :

Jumlah Induk : 3
Jumlah Anak Induk 20 : 3

c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL_13_103112400081_StrukturDataa\output>
```

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita membuat dan mengelola struktur data *Multi Linked List* (list bertingkat) menggunakan *Doubly Linked List*, di mana setiap data induk dapat memiliki banyak data anak, serta mendukung operasi dasar seperti alokasi, pencarian, penambahan, penghapusan, dan penampilan data.

Program ini bertujuan untuk mengimplementasikan **Multi Linked List** dalam bahasa C++, yaitu struktur data yang terdiri dari:

- **List Induk** (parent)
- **List Anak** (child) yang dimiliki oleh setiap induk

Setiap list (induk dan anak) menggunakan konsep **Doubly Linked List**, sehingga setiap elemen memiliki pointer next dan prev.

Program ini memungkinkan pengguna untuk:

1. Membuat list induk dan list anak
2. Menambahkan data induk (insert first, last, after)
3. Menambahkan data anak ke induk tertentu
4. Mencari data induk dan anak
5. Menghitung jumlah data induk dan jumlah anak dari induk tertentu
6. Menampilkan seluruh data multi linked list secara terstruktur

Unguided 2

Coding:

```
#include <iostream>
using namespace std;

#define Nil NULL

struct mahasiswa {
    string nama;
    string nim;
    char jenis_kelamin;
    float ipk;
};
typedef mahasiswa infotype;

struct ElmList;
typedef ElmList* address;

struct ElmList {
    infotype info;
    address next;
};

struct List {
    address First;
};

void createList(List &L) {
    L.First = Nil;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = Nil;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = Nil;
}

void insertFirst(List &L, address P) {
    if (L.First == Nil) {
        L.First = P;
        P->next = P;
    } else {
        address last = L.First;
```



```

        while (last->next != L.First) {
            last = last->next;
        }
        P->next = L.First;
        last->next = P;
        L.First = P;
    }
}

void insertLast(List &L, address P) {
    if (L.First == Nil) {
        insertFirst(L, P);
    } else {
        address last = L.First;
        while (last->next != L.First) {
            last = last->next;
        }
        last->next = P;
        P->next = L.First;
    }
}

void insertAfter(List &L, address Prec, address P) {
    if (Prec != Nil) {
        P->next = Prec->next;
        Prec->next = P;
    }
}

void deleteFirst(List &L, address &P) {
    if (L.First != Nil) {
        if (L.First->next == L.First) {
            P = L.First;
            L.First = Nil;
        } else {
            address last = L.First;
            while (last->next != L.First) {
                last = last->next;
            }
            P = L.First;
            L.First = P->next;
            last->next = L.First;
            P->next = Nil;
        }
    }
}

void deleteAfter(List &L, address Prec, address &P) {
    if (Prec != Nil && Prec->next != L.First) {
        P = Prec->next;
    }
}

```

```

        Prec->next = P->next;
        P->next = Nil;
    }
}

void deleteLast(List &L, address &P) {
    if (L.First != Nil) {
        if (L.First->next == L.First) {
            P = L.First;
            L.First = Nil;
        } else {
            address last = L.First;
            address prec = Nil;
            while (last->next != L.First) {
                prec = last;
                last = last->next;
            }
            P = last;
            prec->next = L.First;
            P->next = Nil;
        }
    }
}

address findElm(List L, infotype x) {
    if (L.First != Nil) {
        address P = L.First;
        do {
            if (P->info.nim == x.nim) {
                return P;
            }
            P = P->next;
        } while (P != L.First);
    }
    return Nil;
}

void printInfo(List L) {
    if (L.First != Nil) {
        address P = L.First;
        cout << "\nIsi Data Mahasiswa\n";
        cout << "=====\n";
        do {
            cout << "Nama : " << P->info.nama << endl;
            cout << "NIM : " << P->info.nim << endl;
            cout << "L/P : " << P->info.jenis_kelamin << endl;
            cout << "IPK : " << P->info.ipk << endl;
            cout << "-----\n";
            P = P->next;
        }
    }
}

```

```

    } while (P != L.First);
  } else {
    cout << "List kosong\n";
  }
}

address createData(string nama, string nim, char jenis_kelamin, float ipk)
{
  infotype x;
  address P;

  x.nama = nama;
  x.nim = nim;
  x.jenis_kelamin = jenis_kelamin;
  x.ipk = ipk;

  P = alokasi(x);
  return P;
}

int main()
{
  List L;
  address P1 = Nil;
  address P2 = Nil;
  infotype x;

  createList(L);

  cout << "coba insert first, last, dan after\n";

  P1 = createData("ALI", "04", 'l', 4.0);
  insertFirst(L, P1);

  P1 = createData("Bobi", "06", 'l', 3.45);
  insertLast(L, P1);

  P1 = createData("Cindi", "02", 'l', 3.71);
  insertFirst(L, P1);

  P1 = createData("Hilmi", "01", 'l', 3.3);
  insertFirst(L, P1);

  P1 = createData("Gita", "07", 'p', 3.75);
  insertLast(L, P1);

  x.nim = "07";
  P1 = findElm(L, x);
  P2 = createData("Cindi", "03", 'p', 3.5);
  insertAfter(L, P1, P2);
}

```

```

x.nim = "02";
P1 = findElm(L, x);
P2 = createData("Hilmi", "08", 'p', 3.3);
insertAfter(L, P1, P2);

x.nim = "04";
P1 = findElm(L, x);
P2 = createData("Eli", "05", 'p', 3.4);
insertAfter(L, P1, P2);

printInfo(L);

return 0;
}

```

Hasil Coding:

```

c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL_13_103112400081_StrukturData\output>.\Unguided2.exe
coba insert first, last, dan after

Isi Data Mahasiswa
=====
Nama : Hilmi
NIM : 01
L/P : l
IPK : 3.3
-----
Nama : Cindi
NIM : 02
L/P : l
IPK : 3.71
-----
Nama : Hilmi
NIM : 08
L/P : p
IPK : 3.3
-----
Nama : Ali
NIM : 04
L/P : l
IPK : 4
-----
Nama : Eli
NIM : 05
L/P : p
IPK : 3.4
-----
Nama : Bobi
NIM : 06
L/P : l
IPK : 3.45
-----
Nama : Gita
NIM : 07
L/P : p
IPK : 3.75
-----
Nama : Cindi
NIM : 03
L/P : p
IPK : 3.5
=====

```

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita membuat dan mengimplementasikan struktur data Circular Linked List (ADT List Sirkular) menggunakan bahasa C++ untuk mengelola data mahasiswa.